AquaScan: An Offline-First Mobile System for Freshwater Fish Disease Classification with Uncertainty Gating and Retrieval-Grounded Assistance

Rachit Bhusal
Computing Sciences Division
Arkansas Tech University
Russellville, AR, USA
rachitbhusal@gmail.com

Robin Ghosh
Dept. of Engineering and Computing Sciences
Arkansas Tech University
Russellville, AR, USA
rghosh@atu.edu

***Abstract*— Fish disease is still judged at the pond edge by eye, and a wrong call costs stock and time. We describe AquaScan, a working system that turns a seven-class freshwater fish disease classifier into something a farmer can actually use, and we are careful to separate what we measured from what we did not. The classifier is a 2.20-million-parameter convolutional network trained on the public South Asian freshwater fish disease corpus, with one preprocessing routine shared byte-for-byte by training, evaluation, and serving. On the distributed `Test/` directory it reaches 81.29% top-1 and 95.14% top-3 accuracy at a macro F_1 of 0.812 (95% bootstrap CI $[0.782, 0.840]$), beating an earlier checkpoint by 11.86 accuracy points; because per-image logs were lost we cannot run McNemar's test exactly, but the marginals alone bound it at $p \leq 7.9 \times 10^{-6}$. We then audited the corpus, and the result reframes that headline. All 700 images in `Test/` are byte-identical copies of images in `Train/`, so the nominal 2 450-file dataset holds 1 747 distinct photographs and any score on that directory measures recall of fitted data. The 63.71% we obtain on an internal held-out split is the only generalisation estimate this study can defend. Around the classifier sit an ImageNet fish-presence gate, a joint confidence–entropy rejection rule, and a retrieval-augmented assistant that answers follow-up questions from a local 36-chunk aquaculture index through a quantised on-device language model, with retrieval conditioned on the prediction. Baseline comparisons, gate accuracy, and retrieval quality are given as protocols and reported as outstanding, not estimated.**

Index Terms— fish disease classification, convolutional neural networks, aquaculture, dataset duplication, out-of-distribution rejection, retrieval-augmented generation, uncertainty estimation, mobile deployment

I. Introduction

Inland aquaculture supplies a large share of the animal protein eaten in South Asia, and disease outbreaks are among the most damaging shocks that supply absorbs year after year [1]. The warning signs are visible long before a laboratory can confirm anything: haemorrhagic patches, eroded fins, inflamed gills, fungal growth, white caudal lesions. They are also easy to mix up. A farmer who reads a parasitic infestation as a bacterial one loses the treatment window and pays for the wrong intervention on top of it.

Convolutional networks suit this decision well. Working on a seven-class South Asian freshwater corpus, Tamut *et al.* [2] showed that a compact three-block network separates the categories, reporting 99.71% accuracy at a test loss of 0.0119. That tells us the visual problem is learnable, and it gives us an architecture to start from. It tells us nothing about what a farmer holding a phone will actually experience.

Three gaps sit between that benchmark and a usable tool. The first is *evidential*. A classifier is worth only as much as the split it was scored on, and the audit in Section III finds that the public corpus contains no test image that is not already in its training directory. The second is *behavioural*. A seven-way softmax is a closed set: photograph a bucket and it will still name a disease. Section IV describes the two-stage service that catches those inputs before the classifier sees them. The third is *interpretive*. A label and a percentage do not tell anyone what to do next, and a general-purpose chatbot will answer the follow-up from parametric memory with nothing to check it against. Section V describes an assistant that answers from a curated index plus the current prediction, runs on the operator's own machine, and shows the passages it used.

We contribute five things. A duplicate-aware re-reading of a widely used public corpus, with the audit that supports it. A classifier trained under a single shared preprocessing contract and reported with bootstrap and Wilson intervals, a full confusion analysis, and a distribution-free significance bound against an earlier checkpoint. A deployed two-stage service and cross-platform client in which gating, rejection, and confidence tiering are visible parts of the product rather than internal details. A fully local retrieval-augmented assistant whose retrieval query is conditioned on the classifier output. And, throughout, a clean line between what we measured and what still needs doing. Fig. 1 shows how the pieces connect.

II. Related Work

Reported accuracies for image-based fish health assessment span a very wide band, mostly because dataset size, class def-

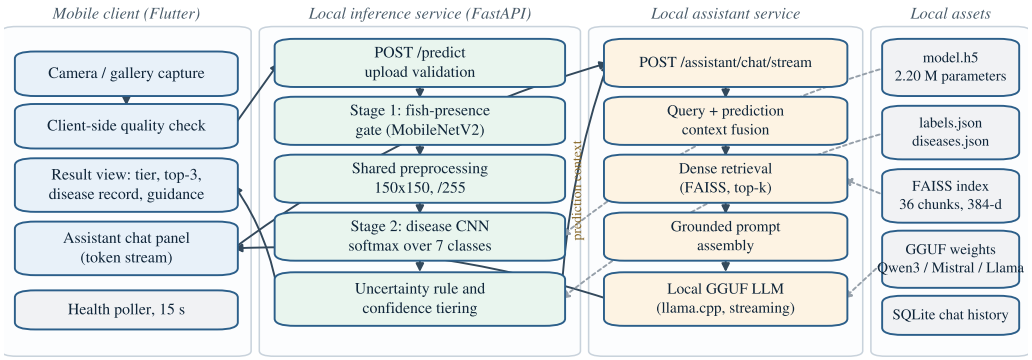


Fig. 1. AquaScan system architecture. The assistant path reuses the classifier output as retrieval context, and every model and data asset resides on the operator's machine.

inition, and split policy differ from paper to paper. Tamut *et al.* [2] trained a three-block network from scratch on the seven categories used here for 50 epochs at batch size 32 with Adam [3], reporting 99.71% accuracy, and also report a ResNet50 [4] transfer trial reaching only 70.83% validation accuracy; they identify bacterial gill and bacterial red disease as the pair most often confused, an observation Section E revisits. Ahmed *et al.* [5] compared four backbones on Bangladeshi aquaculture imagery and reported 82.51% test against 95.51% validation accuracy — a gap that is itself an argument for reporting held-out figures — and Biswas *et al.* [6] evaluated transfer-learning and fusion approaches on the same problem. Every one of these figures rests on the same assumption: that the evaluation images were not seen during fitting. The assumption fails quietly and it fails often. Near-duplicates across the CIFAR splits inflate apparent accuracy [7], standard corpora carry large-scale semantic redundancy [8], and test-set defects have been shown to reorder model rankings outright [9]. Checking for duplication is a precondition for evaluation, not an appendix to it.

Two further literatures shape the deployed system. A softmax cannot express "none of these"; maximum softmax probability is the baseline out-of-distribution detector [10], Shannon entropy [11] supplies a complementary uncertainty measure, and modern networks are systematically overconfident [12]. Separately, retrieval-augmented generation [13] conditions a language model on passages fetched at inference time, making an answer's sources inspectable and the knowledge base correctable without retraining; practical systems pair a sentence encoder [14], [15] with a vector index [16], and retrieval quality, grounding, and hallucination rate require separate measurement [17], [18].

III. Model Development

A. Dataset and Content Audit

AquaScan is trained on the public "Freshwater Fish Disease Aquaculture in South Asia" corpus [19], distributed as `Train/` and `Test/` directories with one subdirectory per class: bacterial red disease, bacterial aeromoniasis, bacterial gill disease, fungal saprolegniasis, healthy fish, parasitic disease, and viral white tail disease. The class-to-index mapping is pinned in a version-controlled `labels.json` read by training, evaluation, and serving alike, and the service refuses to start if the loaded network's output width disagrees with it, removing an entire class of silent label-permutation faults. Counting files gives 2 450 readable images: 1 750 in `Train/` at 250 per class, 700 in `Test/` at 100 per class, a class-imbalance ratio of 1.00, and nothing unreadable. Before trusting that split we audited its contents, hashing every image twice — SHA-256 over the raw bytes, and a 64-bit difference hash over an EXIF-corrected 9×8 greyscale reduction.

Table I
Content Audit of the Distributed Corpus

Property	Value
Readable files (`Train/` 1 750, `Test/` 700)	2 450
Unreadable files / train imbalance ratio	0 / 1.00
Exact (SHA-256) duplicate groups, all cross-split	698
`Test/` files with a byte-identical `Train/` twin	700 / 700
Groups containing ≥ 2 `Train/` files	3
Distinct images after collapsing duplicates	**1 747**
Cross-split near-duplicate pairs (Hamming ≤ 4)	208
distinct `Test/` images involved	181
Duplicate groups / pairs with label conflicts	1 / 5

The result (Table I) left little room for interpretation. There are 698 exact duplicate groups. Every one of them crosses the train/test boundary, and all 700 files in `Test/` belong to one. Put plainly: the distributed test directory contains no image that is not already sitting in the training directory, byte for byte. Collapse the groups and 1 747 distinct photographs remain, meaning 703 of the 2 450 files are copies. Duplication *inside* `Train/` is a different story — three groups covering six images — so the internal validation split described in Section B is essentially clean. Beyond the exact matches, 208 cross-split pairs agree at Hamming distance ≤ 4 without being byte-identical, covering 181 distinct test images. One exact group and five near-duplicate pairs carry contradictory labels; in the exact case the same photograph is filed under both *bacterial gill disease* and *parasitic disease*.

This cuts the same way for us as for everyone else who has

used this corpus. A model fitted on `Train/` has already seen every image in `Test/`, so a score on that directory measures how faithfully the model reproduces labels for photographs it was fitted on. It says nothing about new fish, new farms, or new cameras. We still report those numbers in Section C, for two reasons: they are what the deployed artefact actually produces, and they are the only figures comparable with published work. But we label them in-sample every time they appear.

B. Preprocessing, Architecture, and Training

Preprocessing lives in one module, called identically by the training data loader, the offline evaluator, and the request handler. This is not tidiness for its own sake: a mismatch between the fitting path and the serving path cost us more accuracy during development than any modelling decision, and centralising the transform is the structural fix rather than a workaround. The pipeline applies the EXIF orientation transform, converts to three-channel RGB, resizes to 150×150 with Lanczos resampling, and scales to $[0, 1]$; images whose short side falls below 64 pixels are flagged rather than refused. Augmentation runs only during training and only *after* the shared transform, so it can never become a hidden difference between fitting and serving: horizontal flip, rotation to $\pm 30^{\circ}$ with reflection fill, translation to 15% of each axis, zoom within $\pm 20\%$, and contrast and brightness jitter of $\pm 20\%$. Vertical flips and hue shifts are excluded because fish orientation and lesion colour carry class information.

The deployed network follows the three-block design of [2], adopted because it is validated on this exact taxonomy and because 2.20 M parameters keep the serialised weights small enough to ship offline. Its structure, recovered from the serialised artefact rather than from documentation, is Conv2D(5×5, 128) $\rightarrow$ Conv2D(3×3, 64) $\rightarrow$ Conv2D(3×3, 32), each followed by 2×2 max-pooling, batch normalisation [20], and 25% dropout [21], then Flatten(8 192) $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(7, softmax), with an L2 penalty of 10^{-4} on every kernel; the total is 2 202 087 parameters, of which 2 201 639 are trainable. Three things about this design are worth flagging, because they are what Section D is really testing. Feature width *decreases* with depth, which is the reverse of the usual convention; the wide 5×5 first layer suits the coarse, high-contrast lesion patterns that separate these classes, and narrowing the later blocks keeps the $73 \times 73 \times 128$ intermediate map affordable. With fewer than 1 500 effective training images, the regularisation density is not optional. And the parameter budget is lopsided: the 256-unit layer after flattening alone carries 2 097 408 parameters, 95.2% of the model, where global average pooling would need 8 448. Most of this network is one fully connected layer.

Training holds out 15% of `Train/` per class under a fixed seed of 42 and never touches `Test/`. The deployed checkpoint used batch size 32 over 60 epochs at an initial learning rate of 8×10^{-4}, AdamW [22], and categorical cross-entropy with label smoothing 0.05 [23] on Keras 3.14; early stopping monitored validation loss with patience 15 and restored the best weights, and the learning rate was halved on plateau with patience 5, falling four times to 5×10^{-5}. Label smoothing was introduced because the raw network was confidently wrong on ambiguous bacterial cases, a direct usability problem for a tool that displays a confidence percentage; AdamW decouples weight decay from the adaptive step, although the deployed checkpoint predates the change that disables kernel L2 when AdamW is active and therefore carries both; and class weights use the effective-number formulation [24], which on this balanced corpus evaluates to 0.998 for six classes and 1.012 for the seventh and is retained for the imbalanced field data anticipated in Section VI. Both callbacks monitor loss rather than accuracy, because under label smoothing the loss is the more sensitive indicator of when calibration stops improving.

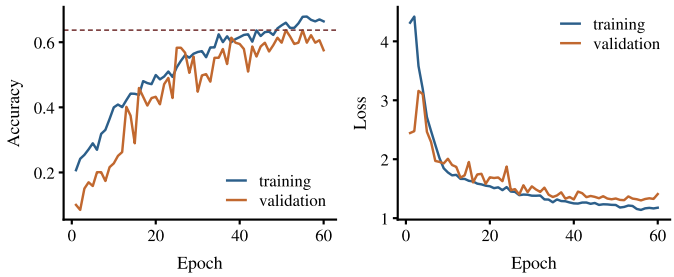


Fig. 2. Training and validation trajectories over 60 epochs. The dashed line marks the best validation accuracy, 63.71% at epoch 51; the restored checkpoint is the one minimising validation loss at epoch 56.

Best internal validation accuracy is 63.71% at epoch 51, with the minimum validation loss of 1.3010 arriving at epoch 56 (Fig. 2). That sits well below the 81.29% we get on the distributed test directory, and at first glance it looks like something is broken. The audit explains it. The validation split holds images the network never saw; the test directory holds images it saw repeatedly. We therefore treat 63.71% as the generalisation estimate this study can support, and 81.29% as a reproduction measure. The gap between the two curves, opening under augmentation and dropout that are already aggressive, points at training-set diversity rather than model capacity as the thing holding this model back.

C. Evaluation and Comparison Against a Previous Checkpoint

The offline evaluator loads the model through the same loader the service uses, applies the shared transform, and predicts over all 700 images. Table II collects the results. Aggregate intervals come from 10 000 bootstrap resamples [25], and we cross-checked the accuracy interval with the Wilson score method [26]; the two agree to within three thousandths. Class supports are exactly equal here, so macro and weighted averages coincide and balanced accuracy equals top-1 accuracy. The most useful number in the table is the 13.9-point gap between top-3 and top-1: in 95.14% of cases the correct condition is somewhere in the three-item differential the interface shows, even when the top-ranked label is wrong. That gap is the reason the interface never displays a bare verdict.

An earlier checkpoint of the same architecture, scored on the same 700 images through the same pipeline, achieved 69.43%

Table II
IN-SAMPLE PERFORMANCE ON THE DISTRIBUTED TEST/ DIRECTORY ($N = 700$; PER-CLASS $n = 100$)

Aggregate metrics with 95% intervals		
Top-1 accuracy (bootstrap)	0.8129	$[0.7829, 0.8414]$
Top-1 accuracy (Wilson)	0.8129	$[0.7823, 0.8400]$
Top-3 accuracy	0.9514	—
Macro precision	0.8223	$[0.7938, 0.8494]$
Macro recall / balanced acc.	0.8129	$[0.7839, 0.8406]$
Macro F_1	0.8125	$[0.7820, 0.8397]$
Matthews correlation [27]	0.7834	$[0.7498, 0.8160]$
Cohen's κ [28]	0.7817	$[0.7469, 0.8146]$
Log loss	0.7830	—

Per-class results, F_1 with 95% bootstrap CI			
Class	**Prec.**	**Rec.**	F_1
Bacterial red disease	0.869	0.730	0.793 [0.72, 0.85]
Bacterial aeromoniasis	0.880	0.730	0.798 [0.73, 0.86]
Bacterial gill disease	0.911	0.820	0.863 [0.81, 0.91]
Fungal saprolegniasis	0.824	0.840	0.832 [0.77, 0.88]
Healthy fish	0.798	0.950	0.868 [0.82, 0.91]
Parasitic disease	0.690	0.890	0.777 [0.71, 0.83]
Viral white tail disease	0.785	0.730	0.756 [0.68, 0.82]
Macro average	**0.822**	**0.813**	**0.812**

top-1 accuracy (486 correct), 92.00% top-3 accuracy, and macro F_1 0.698, against 569 correct for the deployed checkpoint: an improvement of 11.86 accuracy points and 0.114 macro F_1, with Wilson intervals $[0.659, 0.727]$ and $[0.782, 0.840]$. Both were scored on the same images, so an unpaired two-proportion test would be wrong here and McNemar's test [29], [30] is the right instrument. Computing it exactly needs the per-image agreement table, and we did not keep per-image logs for the earlier checkpoint — an omission we have since fixed. The marginals still pin the answer down. Write b for the images the deployed model gets right and the earlier one does not, and c for the converse. Then $b - c = 83$, $b \le 214$, and $c \le 131$, so $b + c \le 345$. Because $\chi^2 = (b-c)^2/(b+c)$ decreases in $b + c$ at fixed $b - c$, every agreement table consistent with what we observed satisfies

$$\chi^2 \;=\; \frac{(b-c)^2}{b+c} \;\ge\; \frac{83^2}{345} \;=\; 19.97, \qquad (1)$$

which gives $p \le 7.9 \times 10^{-6}$ on one degree of freedom. The improvement holds whichever table actually obtained, so nothing is lost by not knowing it. Per-image probability vectors are now written to disk, and future comparisons will use the exact test.

D. Baseline Architectures

We have not compared this classifier against modern backbones on this corpus, so no such comparison appears here. The trainer implements an ImageNet-pretrained [31] MobileNetV2 [32] transfer path — frozen-backbone warm-up then low-learning-rate fine-tuning of the final 30 layers — but no evaluation artefact exists for it, so no number for it is stated.

The comparison is nonetheless well motivated, and we specify it so that it is reproducible (Table III). ResNet50 [4] is the high-capacity reference and the one for which a prior data point exists on this taxonomy [2], making it a control for whether depth alone helps at this data scale; EfficientNetB0 [33] probes whether compound scaling recovers accuracy at a fraction of that budget; MobileNetV2 [32] is the deployment-relevant candidate, already present as the fish-presence gate; and a lightweight vision transformer [34], [35] tests whether global attention helps for lesions distributed across the body. The protocol is fixed in advance: the deduplicated group-disjoint split of Section VI, the shared transform, identical augmentation and optimisation policy, and three fixed seeds per architecture, reporting top-1 and top-3 accuracy, macro precision, recall and F_1, parameter count, serialised size, and median CPU latency, with bootstrap intervals and pairwise McNemar tests [29] against the deployed network. An architecture is promoted only if it improves accuracy and macro F_1 without an unacceptable per-class recall regression and without degrading calibration [12].

Table III
PROPOSED BASELINE COMPARISON. PARAMETER COUNTS ARE PUBLISHED ARCHITECTURAL PROPERTIES AT 224^2 RESOLUTION, NOT MEASUREMENTS FROM THIS STUDY; ACCURACY COLUMNS ARE DELIBERATELY UNPOPULATED

Architecture	**Params (M)**	**Status in this work**
AquaScan CNN (deployed) [2]	2.20	Evaluated, Table II
MobileNetV2 [32]	3.4	Implemented, not evaluated
EfficientNetB0 [33]	5.3	Not implemented
Lightweight ViT [34]	5.7	Not implemented
ResNet50 [4]	25.6	Not implemented

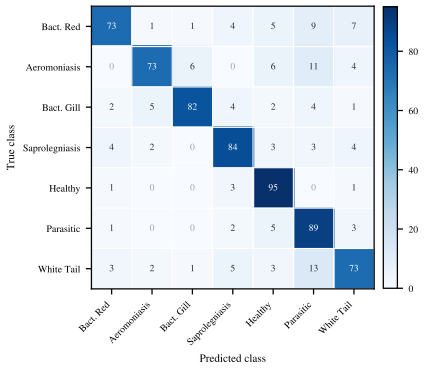


Fig. 3. Confusion matrix on the 700-image test directory; rows are true classes, columns predicted. The dominant off-diagonal mass is the column under *parasitic*, which absorbs 40 of the 131 errors.

E. Confusion Structure and Error Analysis

The model gets 131 of 700 images wrong, and those errors are not spread evenly. Fig. 3 gives the full matrix and Fig. 4 separates, for each class, the errors it loses as a true class from the errors it collects as a predicted one. Four patterns account for most of the mass, and each has a different cause.

Parasitic disease behaves as an attractor. Its column collects 40 misclassifications from the other six classes — 13 from viral white tail, 11 from aeromoniasis, 9 from bacterial red — which is 31% of all errors and the widest bar in Fig. 4. That is why its recall of 0.89 sits alongside a precision of 0.69, the lowest in the model. The cause is a lopsided decision boundary, not a short-

age of examples: parasitic is where the network goes when it is unsure, and going there is cheap. Visible infestation shows up as small, scattered, locally high-contrast marks, and both haemorrhagic spotting and early white-tail lesions produce the same low-level signature. At 150×150 input the texture-scale cues that would separate them have already been resampled away.

Viral white tail disease is the weakest class ($F_1 = 0.756$), losing 13 images to parasitic and 5 to fungal saprolegniasis. The fungal confusion at least makes sense on inspection: both present as pale discoloration of the caudal region, and telling a white lesion from a white fungal mat is a fine-texture call that survives neither the resize nor the brightness jitter we apply during training.

The two non-gill bacterial classes are under-predicted symmetrically, both at recall 0.73 against precision near 0.87, with errors scattering widely rather than concentrating on each other. This differs from [2], where gill-versus-red confusion dominated: here bacterial gill disease is the strongest bacterial class ($F_1 = 0.863$), plausibly because gill pathology is localised to a consistent anatomical region and is therefore learnable as a position-anchored feature, whereas the other bacterial lesions may appear anywhere on the body.

Healthy fish is over-predicted: recall 0.95 against precision 0.798, which means 24 diseased fish get labelled healthy. We regard this as the most damaging error in the matrix. A wrong disease name invites a second opinion; a false clean bill of health ends the investigation. The likely mechanism is mild or early presentation, where the lesion covers a small fraction of the frame and the image sits nearer the healthy manifold than its own class centroid.

Two caveats bound all of this. Every evaluation image was present during fitting, so these 131 are cases the network cannot reproduce even on data it has seen; on genuinely novel photographs the error set will be larger and probably shaped differently. And since one exact duplicate group and five near-duplicate pairs carry contradictory labels, a handful of these errors cannot be won at all. Going further than description — actually attributing the patterns — needs evidence the current artefacts do not hold: saved probability vectors, gradient attribution [36], and per-image capture metadata.

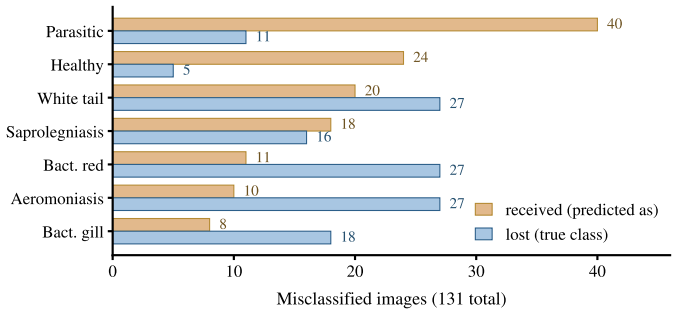


Fig. 4. Where the 131 errors go. For each class, the blue bar counts images of that true class assigned elsewhere and the amber bar counts images of other classes assigned to it. Parasitic disease receives almost four times as many errors as it loses; healthy fish shows the same asymmetry, which is the more dangerous of the two.

IV. Application

A. Service and Two-Stage Prediction Pipeline

The backend is a FastAPI application [37] served by Uvicorn, exposing `GET /health`, `GET /model/info`, `GET /diseases`, and `POST /predict` alongside the assistant routes of Section V; every request is timed and the elapsed milliseconds returned in a response header, which is what lets the client display a per-scan latency figure. Model loading is layered so that the application degrades instead of failing outright: the service prefers a project-level Keras `.h5` artefact, falls back to a bundled copy, and finally to ONNX. Whichever one loads, its output width is checked against `labels.json` at startup, and a mismatch aborts the load rather than quietly serving mislabelled predictions. No inference path makes a network request; the deployed artefacts are a 25.3 MB Keras file, an 8.4 MB ONNX fallback, a 1.5 kB label map, and a 4.7 kB disease record file. Transport validation rejects empty bodies and uploads above 15 MB and checks content types against an allowlist.

1) Fish-presence gate

A closed seven-way classifier will assign a disease to any input [10]. To prevent this, an ImageNet-pretrained [31] MobileNetV2 [32] is run first on a 224×224 resize of the same image. Let $\mathbf{p} \in \Delta^{999}$ be its 1000-way output and $\mathcal{F}$ the set of 16 ImageNet indices corresponding to fish — tench, goldfish, great white shark, tiger shark, hammerhead, electric ray, stingray, barracouta, eel, coho, rock beauty, anemone fish, sturgeon, gar, lionfish, and puffer. The gate computes an aggregate fish score $s_{\text{fish}} = \sum_{c \in \mathcal{F}} p_c$ and admits the image when

$$\arg\max_c p_c \in \mathcal{F} \quad \text{or} \quad s_{\text{fish}} \geq \tau_g, \qquad \tau_g = 0.05. \tag{2}$$

The disjunction is doing real work. Farmed carp and tilapia are not ImageNet categories, so a diseased carp usually spreads modest probability over several fish classes without any single one winning; the aggregate term catches that case, and the $\arg\max$ term catches the unambiguous fish that the aggregate would also admit. The threshold sits three orders of magnitude above the $1/1000$ chance level and is environment-configurable. The gate fails open — if it cannot be loaded, the detector returns "fish present" with an availability flag cleared and the request proceeds — so a malfunction never blocks a legitimate scan, at the cost that unavailability silently removes the protection; the gate is therefore warmed at startup and its status reported by `/health`. On rejection the response names the closest ImageNet label. We report no accuracy for the gate: measuring it requires a labelled fish/non-fish set that this project does not contain, and the protocol is given in Section VI.

2) Uncertainty rejection and confidence tiering

Let $\mathbf{q} \in \Delta^6$ be the seven-way softmax output and $H(\mathbf{q}) = -\sum_k q_k \log q_k$ its Shannon entropy [11], bounded above by $\log 7 \approx 1.9459$. The service declares "No Fish Detected" when

$$\max_k q_k < \tau_c \quad \text{and} \quad H(\mathbf{q}) > \tau_H, \qquad \tau_c = 0.20,\ \tau_H = 1.90. \tag{3}$$

The conjunction is deliberate, and we arrived at it the hard way. A maximum-probability test on its own [10] rejects far too much: chance level for seven classes is $1/7 \approx 0.143$, and phone photographs taken in bad light routinely produce a *correct* top class at 0.30–0.45. That is a usable differential, not a failure, and throwing it away frustrates the user for no benefit. Requiring the distribution to be simultaneously low-confidence and near-uniform restricts rejection to outputs carrying essentially no signal — $\tau_H = 1.90$ is 97.6% of maximum attainable entropy. Surviving predictions are labelled high (≥ 0.70), medium (≥ 0.45), or low confidence, with recommendation text generated per tier: high gives the treatment record directly, medium asks for a confirming photograph from another angle, and low states explicitly that the result is a hint and not a diagnosis. Because softmax outputs are known to be poorly calibrated [12], these thresholds are interface operating points rather than calibrated posteriors. The response also carries the top-3 ranked classes with probabilities, elapsed inference time, the active backend, and any image-quality warning.

B. Mobile Application

The client is a Flutter application [38] (Dart SDK $\geq$ 3.3, Material 3) built from one codebase for Android, iOS, Windows, and the web. Fig. 5 shows it running on an Android handset. Capture goes through `image_picker` with runtime permission handling at quality 90 and a maximum width of 1500 px, and a local decode before upload warns the user below a 200 px short side, above a 3:1 aspect ratio, or below 15 kB. Prediction is a multipart `POST /predict` with a 30 s timeout. A status service polls `/health` every 15 s across four states and disables the scan buttons when the backend is unreachable, which is better than letting a scan fail after the photograph has already been taken. Alongside this the client offers an offline disease library, session-scoped scan history, optional Firebase Google Sign-In with a persistent guest mode, and the assistant panel of Section V.

Two decisions in this client come from measured model behaviour rather than interface convention. The first is that the result screen shows a differential rather than a verdict: top-3 accuracy beats top-1 by 13.9 points, so the second and third ranked classes carry information worth showing, and Fig. 5(c) presents all three with proportional bars. The second is that confidence is displayed, tiered, and acted on. The ring, the badge, and the recommendation text all read from the same thresholds the server uses, so a medium-confidence result looks like one everywhere in the interface. Given that the model over-predicts healthy fish (Section E), hiding uncertainty in the presentation layer would do active harm.

One inconsistency is worth naming rather than cropping out. The in-app copy visible in Fig. 5(b) still describes 81% as accuracy on a held-out test set, which is the framing we used before the audit of Section A. Correcting that string, and reporting the internal-validation figure instead, is part of the deployment work in Section VI. The backend is covered by four `pytest` suites exercising upload validation, health and model metadata, label-map consistency, top-3 ordering, prediction determinism for identical input bytes, and preprocessing parity, and the client carries unit and widget tests for authentication, the prediction service, the assistant controller, and application boot.

V. LLM Integration

A disease label answers one question and immediately raises four more. What causes this? How does it spread? What water conditions favour it? What should I do first? AquaScan handles those with a retrieval-augmented assistant [13] that runs entirely on the operator's machine (Fig. 6).

A. Knowledge Base, Embedding, and Retrieval

The corpus is deliberately small and curated rather than scraped: five hand-written aquaculture notes covering biosecurity, common conditions and differentials, interpretation of model predictions, species and feeding, and water quality (1 913 words in total), plus the application's structured disease record file, the canonical label map, and the project's evaluation protocol note. Structured sources are linearised into field/value prose before indexing, so a disease record becomes a paragraph rather than a JSON fragment, and the ingestion layer also accepts CSV, PDF, and Excel — the path by which farm-specific guidance would be added. Documents are split into 900-character chunks with 140-character overlap at paragraph, sentence, or word boundaries, yielding 36 chunks. Index freshness is managed by content: a manifest records the SHA-256 digest, size, and modification time of every source together with the embedding model and chunking parameters, and any difference triggers an automatic atomic rebuild.

Chunks are embedded with `all-MiniLM-L6-v2` [14], [15] into 384-dimensional ℓ_2-normalised vectors and stored in a FAISS `IndexFlatIP` index [16]. With normalised vectors, inner product is cosine similarity, and at 36 chunks an exact flat index is both faster and simpler than an approximate one — the persisted index is 54 kB. The encoder loads from a local cache; the only point at which the system may contact a network is the first-time download of these weights, after which it is fully offline.

Retrieval is conditioned on the classifier, which is the part of this design we consider load-bearing. Rather than embedding the question on its own, the retriever forms

$$\tilde{q} \;=\; q \oplus d_{\text{name}} \oplus d_{\text{type}} \oplus d_{\text{symptoms}}, \tag{4}$$

where q is the question and the remaining terms come from the prediction context attached to the session, and retrieves the top $k = 4$ chunks by inner product. That is what makes "how

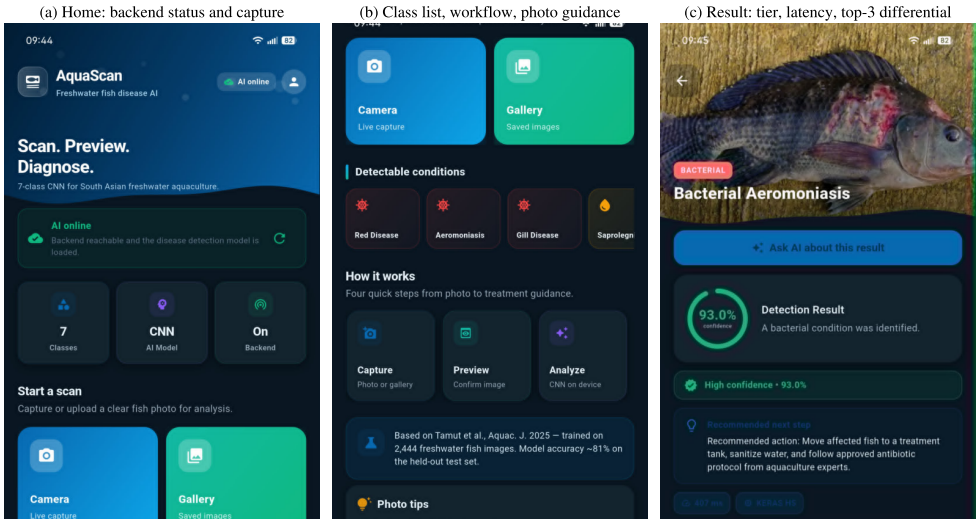


Fig. 5. AquaScan on Android. (a) The home screen surfaces backend reachability before the user commits to a photograph. (b) Detectable conditions, the four-step workflow, and capture guidance. (c) A completed scan: confidence ring and tier badge, the tier-dependent recommendation, chips reporting 407 ms of inference on the Keras backend, and the ranked top-3 differential (93.0%, 4.9%, 1.3%).

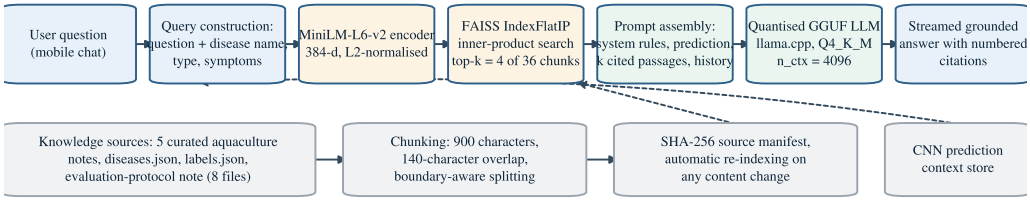


Fig. 6. Retrieval-augmented assistant. The retrieval query concatenates the question with the current classifier output, so a follow-up is resolved against the detected condition. All generation is local.

do I treat this?" answerable at all: the pronoun gets grounded by the classifier output before the query ever reaches the index. With no prediction attached, $\tilde{q}$ collapses to q and the assistant falls back to being a general aquaculture reference. Context enters the session automatically — `/predict` accepts an optional session identifier and on success publishes the full prediction payload to that session's context store, with publication failures logged and swallowed so that an assistant fault can never fail a scan.

B. Prompt Construction and Local Generation

The assembled prompt contains a system instruction, the current prediction serialised as JSON, the k retrieved passages numbered `[1]–[k]` with titles and source paths, up to six turns of history, and the question. The system instruction encodes the project's clinical posture explicitly: retrieved documents are reference data and never instructions; useful passages must be cited by number; absent or conflicting evidence must be stated rather than filled in; a softmax score is a model probability and not clinical certainty [12]; the assistant must not claim to know which image regions drove a prediction; "No Fish Detected" and low-confidence outputs are not diagnoses; antibiotic dosing is never prescribed, because it depends on jurisdiction, species, and withdrawal period; and professional confirmation is recommended before medication or culling. The assistant is an information tool, not a diagnostic authority.

Generation runs through `llama.cpp` [39] on quantised GGUF weights, with three models switchable at runtime — Qwen3-8B (default), Mistral-7B-Instruct-v0.2, and Llama-3.1-8B-Instruct — each at `Q4_K_M` quantisation and 4.1–4.7 GiB on disk. Prompts are rendered with the native control tokens and stop sequences of each family rather than a generic template, and reasoning spans emitted by the Qwen model are stripped by a state machine that tolerates tags split across token boundaries. Decoding uses a 4096-token context, a 256-token output cap, temperature 0.35, top-p 0.90, top-k 40, and repetition penalty 1.12, with prompt length checked against the context window beforehand so that an overlong conversation produces an actionable error rather than a truncation. A low-memory mode keeps quantised weights memory-mapped instead of repacked into committed RAM, which is what allows an 8B model to coexist with the TensorFlow inference process on a 16 GB machine; one model is resident at a time and concurrent requests are refused rather than queued. Answers stream as newline-delimited JSON whose first event carries the model, the retrieved sources, and the active prediction, so provenance is displayed before the first token arrives, and conversations persist in a local SQLite

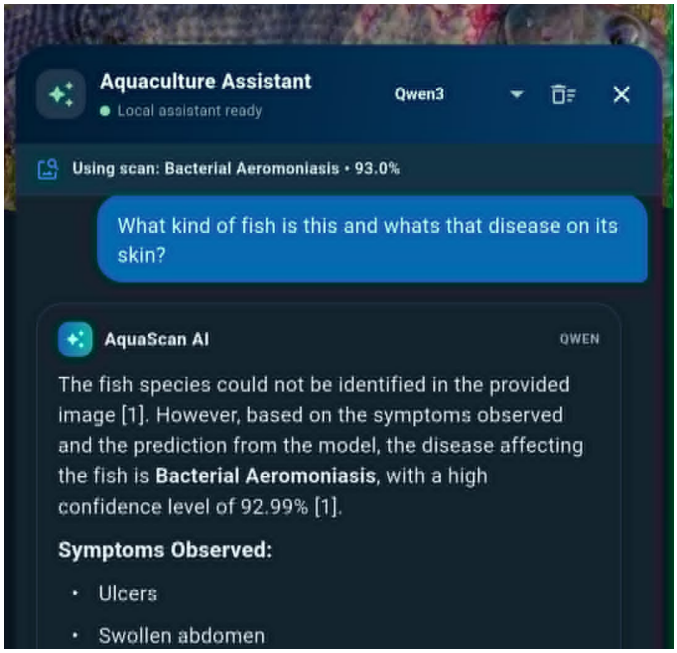


Fig. 7. The assistant answering a follow-up question about the scan in Fig. 5(c). The context bar names the prediction the retrieval query is conditioned on, `[1]` markers point back to retrieved passages, and the reply declines to name the fish species rather than inventing one.

database keyed by session. Three things separate this from a general-purpose chatbot. The answer is drawn from a corpus the operator controls and can extend. The passages behind it are shown next to it. And no image, prediction, or question ever leaves the machine.

Fig. 7 shows one exchange in the deployed client. The header reports the active generator and the scan in context, and the reply carries `[1]` markers back to the retrieved passage. Two details are worth pointing out, both of which follow from the system instruction rather than from luck. The assistant states that it cannot identify the fish species from the image, which is correct — species is not something the classifier predicts and not something the knowledge base contains — rather than guessing a plausible name. And it presents the 93.0% output as a model probability attached to a named condition, then moves on to differential diagnoses instead of closing the question. We show this as an illustration of the intended behaviour, not as evidence of it: one conversation is not a measurement, and Section C sets out what a measurement would require.

C. Evaluation Protocol for the Assistant

We have not evaluated the retrieval or generation components, and we report no numbers for them. What we do have is functional coverage: automated tests verify the streaming contract, session validation, single-flight generation locking, history persistence, prediction-context publication, and index rebuild-on-change. Those establish that the pipeline behaves as specified. They say nothing whatsoever about whether the answers are any good.

The protocol we regard as necessary before any quality claim follows. *Benchmark:* at least 120 aquaculture questions stratified across eight intents — symptoms, causal agents, prevention, treatment and management, water quality, biosecurity, differential diagnosis between visually similar conditions, and interpretation of a model prediction — each annotated by a domain expert with the chunks containing the answer and a reference answer, and including at least 20 questions whose answers are deliberately *absent* from the corpus, since correct refusal is a measurable property the system instruction requires. *Retrieval:* Recall@k and Precision@k for $k \in \{1, 2, 4, 8\}$ against the expert annotations, plus mean reciprocal rank and nDCG [40], where

$$\text{Recall@}k = \frac{|\{\text{queries with a relevant chunk in top-}k\}|}{|\{\text{queries}\}|}. \quad (5)$$

The production setting $k = 4$ over a 36-chunk index must be validated rather than assumed. *Grounding:* each generated claim labelled supported, contradicted, or unsupported with respect to the retrieved passages, giving a grounding rate and an unsupported-claim rate, with citation correctness scored separately as the fraction of `[n]` markers pointing to a passage that actually contains the cited content — a plausible answer with a wrong citation is a specific and serious failure. Automated approximations [17] may be used for iteration, but reported figures should rest on expert adjudication with inter-annotator agreement stated. *Hallucination and safety:* hallucination frequency measured on the unanswerable subset as the rate at which the assistant answers rather than declining [18], and adherence to the safety constraints scored as a separate pass rate. *Latency:* time to first token and total generation time, median and 95th percentile, per model and hardware, since a 4 GiB quantised model on CPU is the binding usability constraint in the field. Until those measurements exist, the assistant is an implemented architecture of unmeasured quality, and we would rather say so than publish a plausible-looking score.

VI. Future Work

The first four items validate claims already made and are ordered by how much they would change this paper's conclusions.

A clean split, and the comparison that depends on it. Nothing else on this list matters until the evaluation set can be trusted. Since the distributed test directory is a strict subset of the training directory, the partition must be rebuilt from the 1 747 distinct images: collapse exact-duplicate groups to one representative, resolve or discard the pairs carrying conflicting labels, cluster near-duplicates at Hamming distance ≤ 4, and split at the level of groups rather than images. The partition should be published alongside the model so subsequent work is comparable. All figures in Section C must then be recomputed, and we expect them to fall; the 63.71% internal validation accuracy is the better prior. On that split the four baselines of Section D should be trained under the protocol specified there, testing in particular whether replacing the 2.10 M-parameter dense projection with global average pooling recovers most of the benefit of a modern backbone at a fraction of its

cost. Field collection should then extend the corpus with expert-verified pond-side photographs under grouped splitting, recording species, farm, device, lighting, and disease stage so that performance can be reported per stratum; active learning [41] would let reviewed misclassifications re-enter training.

Quantitative gate and OOD evaluation. The gate ships in the product and has never been measured. Evaluation needs at least 300 fish photographs spanning the seven classes, farmed species outside the ImageNet fish taxonomy, and adverse conditions (underwater, occluded, at distance, turbid water, held in hand, in a net), and at least 300 non-fish images from the categories the gate will actually meet — pond surfaces, tanks, nets, buckets, feed, aeration equipment, people and hands, scenery, screens, and blank or blurred frames. The gate should then be characterised by precision, recall, specificity, false-positive and false-negative rate, and accuracy with 95% intervals, and by an ROC over τ_g so that the 0.05 operating point is justified rather than asserted [10]. The two disjuncts of (2) should be ablated separately, since our expectation that the $\arg\max$ term alone fails on farmed carp and tilapia is an untested design rationale. The error types are not symmetric — a false negative costs a retake, whereas a false positive delivers a fabricated disease label with a confidence percentage attached — and fail-open behaviour should additionally be measured as an availability rate under memory pressure.

Formal RAG evaluation. The benchmark and metric suite of Section C should be built and run in full across all three configured generators under identical retrieval conditions and decoding parameters, with expert review conducted by aquaculture practitioners rather than by the system's authors. Three ablations matter: retrieval depth k; the prediction-conditioned query of (4) against question-only retrieval, which is the component that most distinguishes this assistant from a generic document chatbot; and a retrieval-disabled condition, since the central claim of the design is that retrieval reduces unsupported claims relative to a parametric-only baseline, and without that ablation the claim remains untested.

Calibration, significance, and attributed error analysis. The evaluation pipeline should persist per-image probability vectors, not only aggregate metrics. That single change replaces the bound of (1) with exact McNemar tests [29], [30], makes expected calibration error and multiclass Brier score [42], [12] computable, and allows errors to be stratified by confidence. Calibration matters directly because the interface exposes confidence and tiers its recommendations on it: if temperature scaling improves reliability, the 0.70 and 0.45 boundaries should be re-derived from the calibrated distribution so that a stated confidence of 0.70 corresponds to roughly a 70% empirical hit rate. Bootstrap resampling [25] should accompany every reported difference. The error patterns of Section E should then be attributed rather than inferred: Grad-CAM [36] would indicate whether the network attends to lesions or to background artefacts, and capture metadata would separate photographic causes from genuine ambiguity. Input resolution should be revisited directly, since the 150×150 input inherited from [2] is the most likely single cause of the parasitic/white-tail and white-tail/fungal confusions.

Deployment. On-device inference through TensorFlow Lite [43] or ONNX Runtime Mobile would remove the local-server requirement, with post-quantisation accuracy loss, latency, and memory footprint reported on low-cost Android hardware; a staged path is realistic — quantised CNN inference first, retrieval second, generation last, since the language model dominates memory. Localisation of the interface and, harder, of the retrieval corpus into Nepali, Bengali, and Hindi is a precondition for use by the farmers this system targets, and cross-lingual retrieval quality must be verified rather than assumed. Optional consented geotagging would allow scans to be aggregated into regional outbreak maps, a capability no individual prediction can provide, noting that the current privacy posture — nothing leaves the machine — is itself worth preserving.

VII. Conclusion

We set out to build a fish disease tool and ended up, along the way, having to re-read the benchmark it was built on. A 2.20 M-parameter convolutional network trained under a shared preprocessing contract reaches 81.29% top-1 and 95.14% top-3 accuracy with macro F_1 0.812 on the distributed test directory, improving on an earlier checkpoint by 11.86 accuracy points at a significance level bounded by $p \leq 7.9 \times 10^{-6}$ without requiring the paired agreement table. A content audit showed that all 700 images in that directory are byte-identical copies of training images, leaving 1 747 distinct photographs in a nominally 2 450-file corpus, which constrains how the 81.29% figure — and every other figure published on this corpus — may be interpreted, and makes the 63.71% internal validation accuracy the honest generalisation estimate available here. The classifier runs behind an ImageNet fish-presence gate and a joint confidence–entropy rejection rule, so a photograph of a bucket returns "No Fish Detected" rather than a disease, and its output is presented as a tiered three-item differential justified by the 13.9-point gap between top-3 and top-1 accuracy, while a retrieval-augmented assistant conditioned on that output answers follow-up questions from cited local passages without contacting a network. We have been equally explicit about what we did not measure. Comparative baselines, gate accuracy, and retrieval and grounding quality appear here as protocols and as outstanding work, not as estimates. Given the duplication finding, rebuilding the evaluation split comes before all of them. On a corpus where the test set is a copy of the training set, a higher number is not a better model.

References

[1] Food and Agriculture Organization of the United Nations. *The State of World Fisheries and Aquaculture 2024: Blue Transformation in Action*. FAO, Rome, Italy, 2024.

[2] Hayin Tamut, Robin Ghosh, Kamal Gosh, and Md Abdus Salam Siddique. Enhancing disease detection in the aquaculture sector using convolutional neural networks analysis. *Aquaculture Journal*, 5(1):6, 2025.

[3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[5] Sabbir Ahmed et al. Enhancing fish disease classification in Bangladeshi aquaculture through transfer learning and LIME. In *Proc. Int. Conf. Smart Electronics and Communication Systems (ICSES)*, 2024.

[6] Subhajit Biswas et al. Empirical evaluation of deep learning techniques for fish disease detection in aquaculture systems. *IEEE Access*, 2024.

[7] Björn Barz and Joachim Denzler. Do we train on test data? Purging CIFAR of near-duplicates. *Journal of Imaging*, 6(6):41, 2020.

[8] Vighnesh Birodkar, Hossein Mobahi, and Samy Bengio. Semantic redundancies in image-classification datasets: The 10% you don't need. *arXiv preprint arXiv:1901.11409*, 2019.

[9] Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[10] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[11] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

[12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proc. Int. Conf. Machine Learning (ICML)*, pages 1321–1330, 2017.

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[14] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.

[15] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[17] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proc. Conf. European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024.

[18] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[19] Subhajit Biswas. Freshwater fish disease aquaculture in south asia. Kaggle Dataset, 2023. Accessed: 2026-08-09.

[20] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proc. Int. Conf. Machine Learning (ICML)*, pages 448–456, 2015.

[21] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[22] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[23] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016.

[24] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277, 2019.

[25] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[26] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[27] Brian W. Matthews. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) - Protein Structure*, 405(2):442–451, 1975.

[28] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[29] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[30] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[31] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[32] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.

[33] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proc. Int. Conf. Machine Learning (ICML)*, pages 6105–6114, 2019.

[34] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[35] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers and distillation through attention. In *Proc. Int. Conf. Machine Learning (ICML)*, pages 10347–10357, 2021.

[36] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, pages 618–626, 2017.

[37] FastAPI Project. FastAPI: Modern Python web framework. Online documentation, 2026. Accessed: 2026-08-09.

[38] Flutter Team. Flutter: Cross-platform UI toolkit. Google, 2026. Accessed: 2026-08-09.

[39] Georgi Gerganov. llama.cpp: LLM inference in C/C++. GitHub repository, 2023. Accessed: 2026-08-09.

[40] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, U.K., 2008.

[41] Burr Settles. Active learning literature survey. In *Computer Sciences Technical Report 1648, University of Wisconsin–Madison*, 2009.

[42] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[43] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Shlomi Regev, Rocky Rhodes, Tiezhen Wang, and Pete Warden. TensorFlow Lite Micro: Embedded machine learning for TinyML systems. In *Proc. Machine Learning and Systems (MLSys)*, volume 3, pages 800–811, 2021.